

Plant Disease Classification

A binary healthy/diseased leaf classifier, from dataset audit to deployment

Author	Krithick Balaji Ramesh
Task	Internship technical assessment
Dataset	PlantVillage (apple subset, colour)
Model	mobilenet_v2, ImageNet-pretrained, fine-tuned
Test accuracy	0.9951 on 611 held-out images
Date	27 August 2026

Abstract

This report documents a binary healthy/diseased image classifier for apple leaves, trained on a four-class subset of the PlantVillage dataset and served through a FastAPI endpoint with a React front end.

The headline result is 99.51% accuracy and 0.9951 macro F1 on a held-out test set of 611 images. That number is easy to obtain on this dataset and is not the interesting part of the work. PlantVillage photographs each physical leaf several times: the 3,171 images used here come from only 505 distinct leaves. Splitting those images at random places near-identical photographs of the same leaf on both sides of the split, and this report shows that under a naive split 100.0% of validation images have a photograph of the same leaf in the training set, against 0.0% for the leaf-grouped split used throughout. The reported figures are therefore honest rather than flattering, and the limitations section is explicit about what they do and do not support.

1. Problem statement

The task brief asks for a deep-learning image classification system that identifies the health condition of a plant leaf from a photograph, classifies it into a disease or health category, and presents the prediction through a web interface. It specifies PlantVillage as the dataset, names MobileNetV2, ResNet18 and EfficientNet as candidate architectures, requires accuracy, precision, recall, F1 and a confusion matrix, and requires the finished system to be deployed on free hosting with a publicly reachable URL.

1.1 An ambiguity in the brief

The brief specifies the class selection twice, and the two statements disagree. The dataset section asks the intern to "select 3-5 suitable classes from the dataset" with "at least one selected class" representing a healthy condition. The task list then says to "select only two classes: (Healthy, Diseased)" and to "display results (healthy/diseased)".

Rather than silently choosing one reading, this project satisfies both. Four source classes were selected from PlantVillage - three apple diseases and apple healthy, which is three to five classes including a healthy one - and those four classes are mapped onto a binary healthy/diseased target for the deployed model, which is what the web interface requirement describes. Evaluation reports the binary metrics the brief asks for and additionally breaks performance down by the four source classes, which is where the informative errors are. The ambiguity is recorded here rather than resolved by assumption.

2. Dataset

PlantVillage is an open dataset of leaf photographs covering 14 crop species and 26 diseases, introduced by Mohanty, Hughes and Salathe (2016). The copy used here is the Hugging Face mirror, which ships the images together with the authors' own train/test split files and, importantly, a leaf-grouping map.

The full archive contains 54,305 images across 38 classes. Counted from the official split files this is 54,305 rather than the 54,306 quoted in the dataset card - a one-image discrepancy that is noted for accuracy and has no bearing on the work.

Class imbalance across the full dataset is severe: the largest class holds 5,507 images and the smallest 152, a ratio of about 36 to one. That is one reason a single crop was selected rather than the whole dataset.

Property	Value
Source	huggingface.co/datasets/mohanty/PlantVillage
Upstream	github.com/spMohanty/PlantVillage-Dataset
Paper	Mohanty, Hughes & Salathe (2016), <i>Frontiers in Plant Science</i>
Licence	CC BY-SA 3.0
Full dataset	54,305 images, 38 classes, 14 crops

3. Selected classes

The four apple classes were selected. Three reasons, in order of importance:

First, every apple image resolves to a real entry in the dataset's leaf-grouping map - 3,171 of 3,171 images, with no fallbacks. Apple is the only crop for which this is true. Because the entire argument of this report rests on grouping images by physical leaf, it matters that the grouping can be verified rather than assumed.

Second, the binary target is close to balanced at 51.9% healthy. Accuracy is therefore a meaningful headline number and no class weighting is required.

Third, four classes with exactly one healthy class matches the brief's stated range directly.

Source class	Images	Binary label
Apple___Apple_scab	630	diseased
Apple___Black_rot	621	diseased
Apple___Cedar_apple_rust	275	diseased
Apple___healthy	1,645	healthy
Total	3,171	

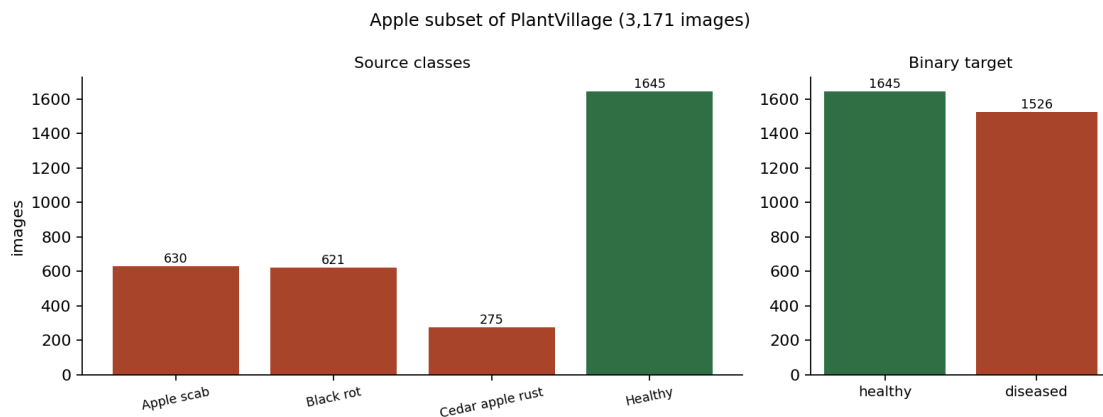


Figure 1. Source class counts and the binary target they collapse into.

3.1 Data quality checks

The subset was profiled before any training, rather than assumed to be clean. All 3,171 images are 256x256, RGB and JPEG; none failed to decode. Seven pairs of byte-identical duplicates were found by MD5, and seven near-duplicate pairs by a 64-bit difference hash at a Hamming distance of three or less. Every one of those pairs falls inside a single leaf group, so the grouped split handles them automatically; no images were removed.

Because every file is the same size, image resolution is reported as a sentence rather than as a histogram.

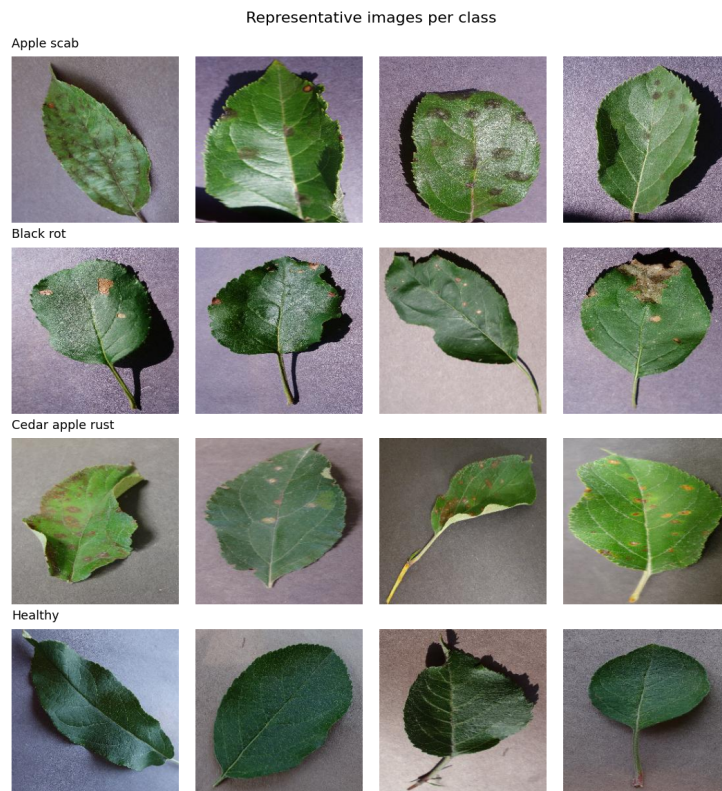


Figure 2. Representative images from each source class.

4. Splitting and data leakage

The test set is PlantVillage's own held-out split, left untouched and used exactly once, after model selection had finished. Validation is carved out of the official training portion using StratifiedGroupKFold grouped on leaf identity and stratified on the four source classes, so that the rarest disease - cedar apple rust, with only 275 images - stays represented in validation. The split is seeded and reproducible.

The official split was audited rather than trusted: no leaf group spans its train/test boundary, for the apple subset or for the dataset as a whole. That is why it is used as the test set here.

Split	Images	Leaves	Healthy	Diseased
train	2,131	340	1,131	1,000
val	429	68	228	201
test	611	97	286	325

4.1 What a naive split leaks

The cost of splitting on images instead of leaves was measured directly, without training anything, by counting how many held-out images have a photograph of the same physical leaf in the training set.

Under the leaf-grouped split used throughout this project, that number is 0 of 429 validation images - 0.0%. Under a naive random image split it is 426 of 426 - 100.0% - and two of those pairs are byte-identical duplicates.

A structural measure is used here in preference to an accuracy comparison, and the reason is worth stating plainly. The binary apple task saturates: validation macro F1 reaches 1.0000 by the third epoch. An accuracy gap between the two split strategies would therefore understate the problem, because there is no headroom in which the optimism can show itself. The leak is present whether or not this particular model is weak enough to expose it, and the count above does not depend on the model at all.

Split strategy	Val images	Sharing a leaf with train	Share
Leaf-grouped (used here)	429	0	0.0%
Naive random image split	426	426	100.0%

Each row is one physical leaf photographed repeatedly

leaf 8.0 (Apple_Frogeye Spot, 21 photos)



leaf 7.0 (Apple_Frogeye Spot, 19 photos)



leaf 108.0 (Apple_scab, 15 photos)



Figure 3. Each row is one physical leaf photographed repeatedly. A random split scatters these rows across train and test.

5. Preprocessing and augmentation

Training images are augmented with a random resized crop to 160 pixels at a scale of 0.7 to 1.0, horizontal and vertical flips, rotation up to 20 degrees, and mild colour jitter. Validation, test and production inputs share a single deterministic path: resize the shorter side to 182, centre crop to 160, and normalise with ImageNet statistics.

Two augmentation choices need justifying.

Both flips are enabled. These are detached leaves photographed against a plain background, so there is no canonical orientation and mirroring is a realistic variation rather than a distortion. This would not be a safe choice for imagery where gravity or plant structure carries information.

Hue jitter is nearly frozen, at 0.02. Leaf colour is the diagnosis in this task - chlorotic yellowing, rust orange, necrotic brown - so a wide hue shift would destroy the signal the model has to learn. Brightness, contrast and saturation are allowed to move further because those vary with photography rather than with pathology.

The deterministic evaluation path is shared with the deployed backend, so the model sees the same pixels in production that it saw during validation. A test asserts that the backend's NumPy reimplementations of that path matches the torchvision original.



Figure 4. One training image under the augmentation pipeline.

5.1 Input resolution

An input resolution of 160 pixels was used rather than the more usual 224. Measured on the four-core CPU this project was trained on, a MobileNetV2 fine-tuning step costs 2.95 seconds per batch of 32 at 224 pixels against 1.90 seconds at 160. The source images are 256x256 close-ups of a single leaf filling the frame, so the additional resolution buys little detail, and the deployment target is a free CPU instance with a fraction of a core allocated.

No full side-by-side comparison of 224 against 160 was run to completion, so no accuracy trade-off is claimed. What can be said is that the 160-pixel model reaches a perfect validation macro F1, leaving 224 no room to score better on that validation set. Whether it would generalise better was not tested.

6. Model and training methodology

All three architectures named in the brief were trained under an identical budget: MobileNetV2, EfficientNet-B0 and ResNet18, each ImageNet-pretrained with only the classifier head replaced.

Training proceeds in two stages. The new head is trained first with the backbone frozen, for two epochs at a learning rate of 1e-3; then the whole network is fine-tuned at 1e-4 with cosine annealing. Freezing first prevents the randomly initialised head from driving large gradients into pretrained features before it has learned anything useful.

Loss is unweighted cross-entropy, since the binary target is close to balanced. The optimiser is AdamW with weight decay 1e-4, batch size 32, for up to six epochs with early stopping on validation macro F1 and a patience of three. Checkpoints are selected on macro F1 rather than accuracy so that neither class can be ignored. The seed is fixed at 42 and the split is reproducible.

Setting	Value
Architecture	mobilenet_v2
Input resolution	160x160
Loss	Cross-entropy (unweighted)
Optimiser	AdamW, weight decay 1e-4
Head LR / fine-tune LR	1e-3 / 1e-4
Scheduler	Cosine annealing during fine-tuning
Batch size	32
Epochs / warm-up / patience	6 / 2 / 3
Checkpoint selection	Best validation macro F1
Seed	42
Hardware	Intel i5-1135G7, 4 cores, CPU only, 16 GB RAM
Framework	PyTorch 2.12 (CPU), torchvision 0.27
Training time	18.6 min
Best epoch	3

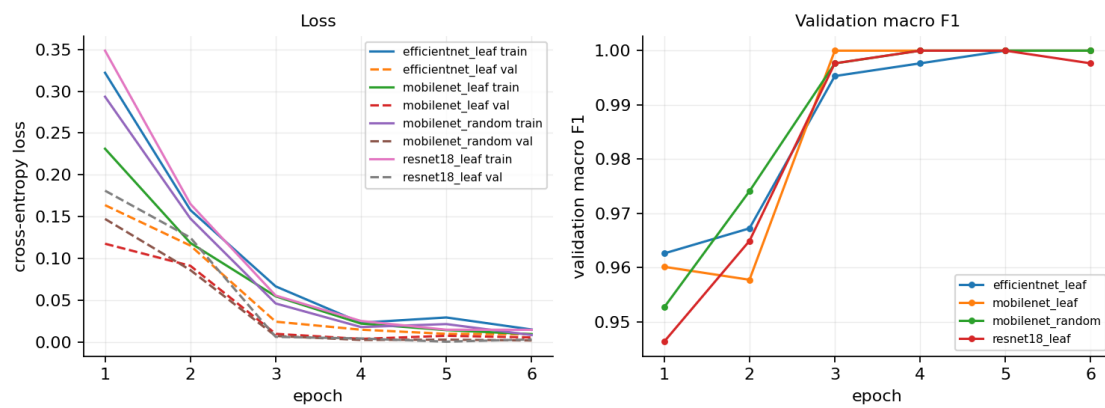


Figure 5. Training and validation curves.

7. Evaluation

All figures below are on the official PlantVillage held-out test split, 611 images from 97 leaves, none of which shares a leaf with any training image. The test set was used once, after model selection had finished on validation.

7.1 Headline metrics

Metric	Value
Accuracy	0.9951
Macro F1	0.9951
Weighted F1	0.9951
Precision (diseased)	1.0000
Recall (diseased)	0.9908
F1 (diseased)	0.9954

7.2 Per-class results

Class	Precision	Recall	F1	Support
healthy	0.9896	1.0000	0.9948	286
diseased	1.0000	0.9908	0.9954	325

7.3 Confusion matrix

	predicted healthy	predicted diseased
actual healthy	286	0
actual diseased	3	322

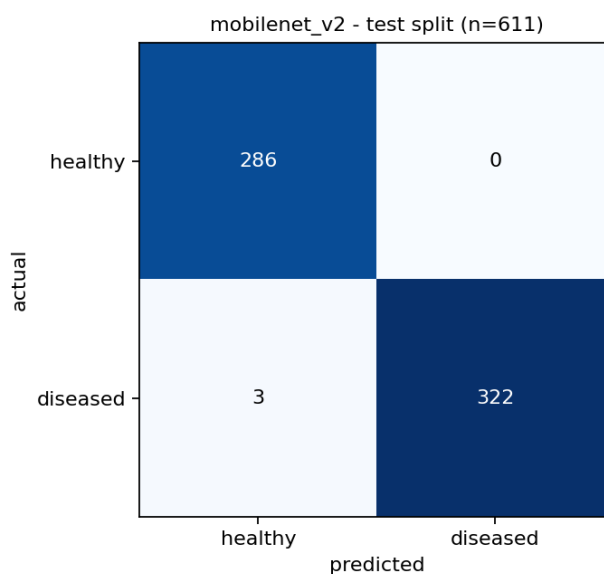


Figure 6. Test-set confusion matrix.

7.4 Performance by original disease

The deployed model answers healthy or diseased, but the four source classes are retained in the manifest so the binary result can be broken down by the disease actually present.

Source class	Images	Correct	Recall	Mean confidence
Apple___Apple_scab	127	124	0.9764	0.9938
Apple___Black_rot	146	146	1.0000	0.9921
Apple___Cedar_apple_rust	52	52	1.0000	0.9999
Apple___healthy	286	286	1.0000	0.9932

7.5 Architecture comparison

Because all candidates saturate the validation set, architecture selection could not be made on validation accuracy in any meaningful sense. The choice was made on deployment cost instead - parameter count, exported model size and CPU inference latency - which is the honest basis given the measurements, and which matches the brief's emphasis on free-tier hosting constraints.

Run	Architecture	Split	Accuracy	Macro F1	Train (min)
efficientnet_leaf	efficientnet_b0	leaf	0.9918	0.9918	49.3
mobilenet_leaf	mobilenet_v2	leaf	0.9951	0.9951	18.6
mobilenet_random	mobilenet_v2	random	0.9967	0.9967	90.1
resnet18_leaf	resnet18	leaf	0.9902	0.9901	18.8

8. Error analysis

The selected model misclassifies 3 of 611 test images. Those mistakes are not independent: all of them are photographs of a single physical leaf, Apple_scab:::94.0. Counted at leaf level - a leaf being missed if any of its photographs is wrong - the model misses 1 of 97 leaves, a leaf-level accuracy of 0.9897. That is the more honest denominator for this dataset, and it is a direct consequence of the same grouping structure that makes the split design necessary.

The leaf in question carries a small number of dark scab lesions near the tip and margin, on tissue that is otherwise unbroken green. Five of its photographs are in the test set; the model calls three of them healthy with high confidence and gets the other two right, one of them only marginally at 0.52 confidence.

One plausible explanation was tested and rejected. Because the lesions sit near the leaf margin, the deterministic centre crop might have been discarding them. Re-running those five images through a full-frame resize that crops nothing changed none of the five predictions. The centre crop is therefore not the mechanism, and the more likely reading is simply that sparse, small lesions on predominantly healthy tissue do not produce enough evidence for this model. That hypothesis was not itself tested further, and is offered as a reading of the evidence rather than a conclusion.

The confidence figures matter more than the error count. Mean confidence on the wrong predictions is 0.9901 and the maximum is 0.9940. A 0.70 confidence threshold - the one the deployed API uses to flag uncertain results - would have caught 0 of 3. The model is confidently wrong, and exposing a confidence score does not protect a user against that. This is a real limitation of the confidence mechanism, not a tuning problem.

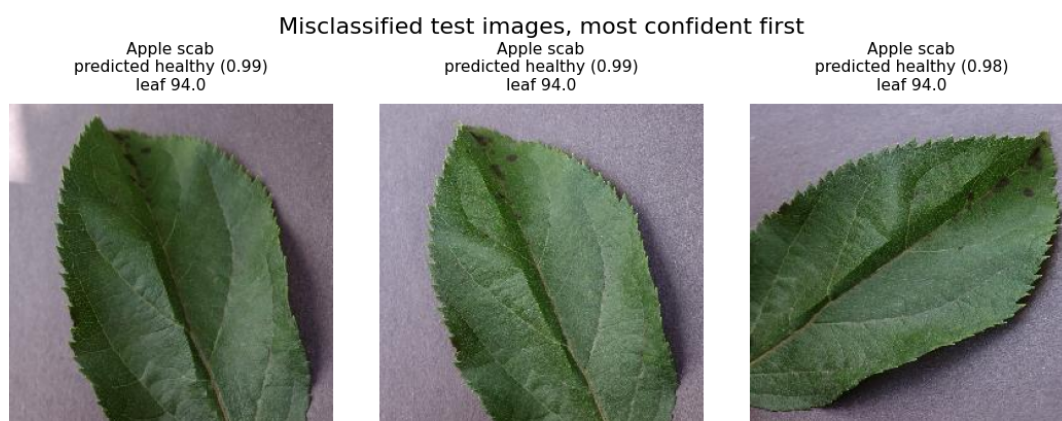


Figure 7. Every misclassified test image, most confident first. All are the same physical leaf.

8.1 Confidence reporting

The API returns the softmax probability of the predicted class and flags anything below 0.70 as low confidence. The threshold is a documented default rather than a tuned operating point, and it is stored in the exported preprocessing configuration rather than hard-coded, so it can be changed without touching code.

No calibration was performed. A softmax score is a model output, not a probability of being correct, and as the error analysis shows this model can be both wrong and confident. The user-facing wording is hedged accordingly: the interface says the model finds patterns consistent with a diseased leaf, not that the leaf is diseased.

9. Backend

The backend is FastAPI served by uvicorn, exposing four endpoints: a service description at the root, a liveness probe at /health, a description of the loaded model at /model-info, and the classifier itself at POST /predict, which accepts a single multipart file field.

Three decisions are worth recording. The model is loaded once, during application startup, because building the inference session costs far more than running it. The service depends on onnxruntime rather than torch, which keeps the deployed image near 150 MB instead of approaching a gigabyte and matters on an instance with 512 MB of memory. Model loading failures are recorded rather than raised, so a missing model produces a degraded health response instead of a container that restarts in a loop.

Uploads are validated on content type, emptiness, byte size against an 8 MB limit, and decodability, with a further guard on total pixel count so that a decompression bomb cannot turn one request into gigabytes of pixels. Internal errors are logged in full and returned to the caller as a generic message.

9.1 API endpoints

Method	Path	Purpose
GET	/	Service description and endpoint list
GET	/health	Liveness probe; reports model load state
GET	/model-info	Architecture, classes, input size, threshold
POST	/predict	Classify an uploaded image (multipart, field file)

9.2 Error responses

Status	Condition
400	Empty, corrupt or undecodable image
413	Over the 8 MB upload limit, or an implausible pixel count
415	Content type is not JPEG, PNG or WebP
422	No file field in the request
503	Model failed to load

10. Frontend

The front end is a small React application built with Vite. The workflow is deliberately linear: select or drop an image, see it previewed, press a single button, read the result. The result panel shows the predicted label, the confidence, both class probabilities and a plain-language message, with an explicit warning when confidence falls below the threshold. A model information panel reports the architecture, class list, input size and threshold, all fetched from the API rather than duplicated in the interface.

The page also documents itself. A short usage guide sits above the upload area, so a first-time visitor learns what the tool expects before choosing a file rather than after a confusing answer; it states plainly that anything other than an apple leaf will still produce a confident-looking result that means nothing. Below the model panel, a tools and techniques reference lists the stack by pipeline stage and explains the five decisions the project turns on, and a short authorship card links to the author's portfolio and resume.

The theme is light, flat and typographic, with no gradients, animation or decorative iconography. A disclaimer states that the system is experimental, that it was trained only on apple leaves photographed under laboratory conditions, and that its output is a prediction rather than a diagnosis.

The interface also accounts for its own hosting. The free backend tier sleeps after fifteen minutes of inactivity and takes about a minute to restart, so the page requests model information on load - which doubles as a wake-up call - shows a notice while it waits, and allows a ninety-second timeout on prediction requests.

11. Libraries and tools

Area	Choice
Training	PyTorch 2.12 (CPU), torchvision 0.27
Data and metrics	NumPy, scikit-learn, Pillow, pandas
Figures	Matplotlib
Serving	ONNX Runtime 1.20 (no torch in the deployed image)
API	FastAPI, uvicorn, Pydantic
Frontend	React 18, Vite 5
Tests	pytest, FastAPI TestClient
Hosting	Render (Docker, backend), Vercel (static, frontend)

12. Deployment

The system is deployed as two pieces: the inference API as a Docker web service on Render's free tier, and the built front end as a static bundle on Vercel.

Hugging Face Spaces was the original target, and the Dockerfile still runs there unchanged. It was ruled out on checking rather than assuming: as of 2026 the Hugging Face documentation states that Gradio and Docker Spaces "require a paid plan to create", with only static Spaces remaining free, and a static Space cannot run the model. This is exactly the kind of change that makes a free-tier assumption stale, and it is recorded here because the same check will need repeating in future.

The constraints of the chosen tier shaped the implementation rather than being discovered after it. The instance provides 512 MB of memory and a fraction of a CPU core, which is why the API serves ONNX and why a single worker is used - each worker would load its own copy of the inference session. The service sleeps after fifteen minutes idle with a cold start of roughly a minute, which the front end handles explicitly. Free instance hours are capped at 750 per month, which is roughly one continuously-running service and no more.

12.1 Free-tier constraints

Constraint	Render free web service
Memory	512 MB
CPU	0.1 CPU
Idle behaviour	Sleeps after 15 minutes
Cold start	About 60 seconds
Monthly allowance	750 instance hours

12.2 Hosted URLs

The deployment artefacts are complete and committed - a Dockerfile, a Render blueprint (`render.yaml`) and step-by-step instructions in `docs/DEPLOYMENT.md` - but the hosted instance is created under the author's own accounts and was not live at the time this document was generated. No URL is quoted here rather than quote one that does not resolve. Rebuild this report with `--app-url` and `--api-url` once deployed.

13. Screenshots

Plant Disease Classification

Upload an apple leaf image to check whether the model reads it as healthy or diseased.

How to use this

1. Choose a leaf photograph — drop it on the upload area or click to browse. JPG, PNG or WebP, up to 8 MB.
2. Check the preview. One apple leaf filling most of the frame, on a plain background, works best.
3. Press "Classify leaf". The first request after a quiet period can take about a minute while the free backend wakes up.
4. Read the result: the predicted label, how confident the model is, and the score for both classes.

The model was trained on apple leaves only. Anything else — another crop, a field photo with a busy background, or a picture that is not a leaf at all — will still produce a confident-looking answer, and that answer means nothing. Confidence below 70% is flagged as inconclusive, but a high score is not a guarantee either.

Upload

Drop a leaf image here, or click to browse

JPG, PNG or WebP, up to 8 MB

Classify leaf

Model

Architecture	Classes	Input size	Confidence threshold
mobilenet_v2	healthy, diseased	160×160 px	70%

Figure 8. Landing view, with the usage guide above the upload area.

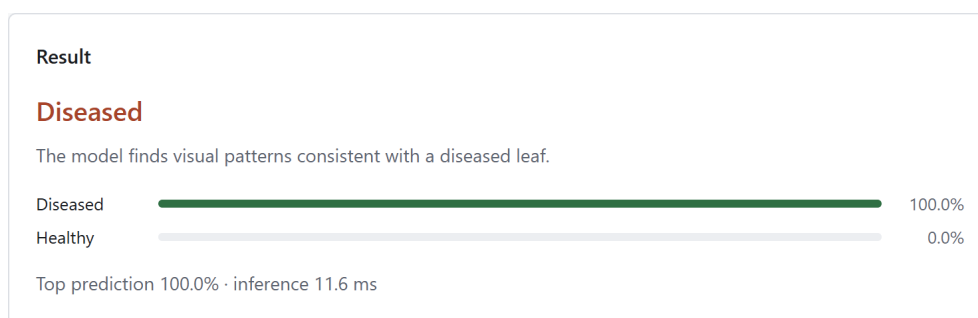


Figure 9. Prediction result, confidence and model panel.

Tools and techniques	
Dataset	PlantVillage (apple subset, 3,171 images) · Hugging Face Hub · Pillow · NumPy · pandas
Data integrity	MDS exact-duplicate detection · 64-bit dHash near-duplicate detection · leaf-group audit
Splitting	scikit-learn StratifiedGroupKFold grouped on leaf identity · fixed seed · disjointness asserted in tests
Model	PyTorch · torchvision · MobileNetV2 / EfficientNet-B0 / ResNet18, ImageNet-pretrained
Training	Two-stage transfer learning (frozen head, then fine-tune) · AdamW · cosine annealing · early stopping on macro F1
Evaluation	scikit-learn metrics · confusion matrix · per-disease breakdown · leaf-level error analysis · Matplotlib
Serving	ONNX Runtime for CPU inference — no PyTorch in the deployed image
Backend	FastAPI · uvicorn · Pydantic · Docker
Frontend	React 18 · Vite
Testing & hosting	pytest · Playwright · Render (API) · Vercel (web app)
Techniques worth calling out	
• Leaf-grouped splitting. The 3,171 images come from only 505 physical leaves. Splitting on images would put photographs of the same leaf on both sides of the split, so the split is grouped by leaf instead. • Transfer learning. ImageNet-pretrained backbone with a replaced classifier head, trained in two stages so the new head does not disturb the pretrained features. • Symptom-preserving augmentation. Flips, small rotations and mild brightness/contrast jitter — but hue is left almost untouched, because leaf colour is the diagnosis. • Macro-F1 checkpointing. The saved model is the one with the best validation macro F1, not the best accuracy, so neither class can be quietly ignored. • Structural leakage measurement. Rather than inferring leakage from an accuracy gap, the pipeline counts how many held-out images share a leaf with training: 0% here against 100% under a naive random split.	

Figure 10. The tools and techniques reference shown in the application itself.

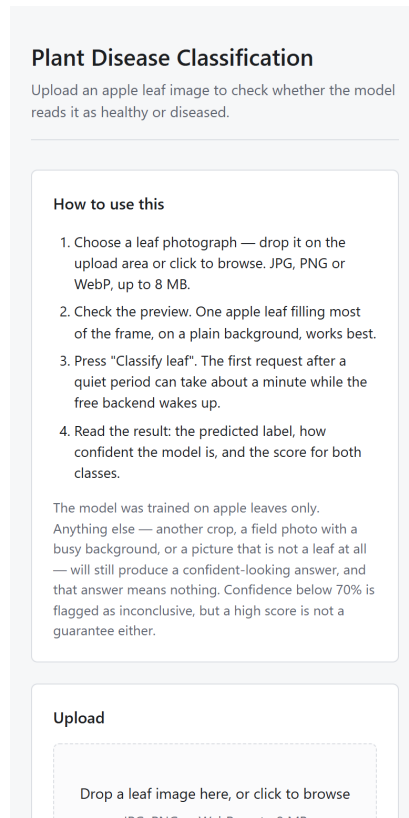


Figure 11. The same page at a 390px phone viewport.

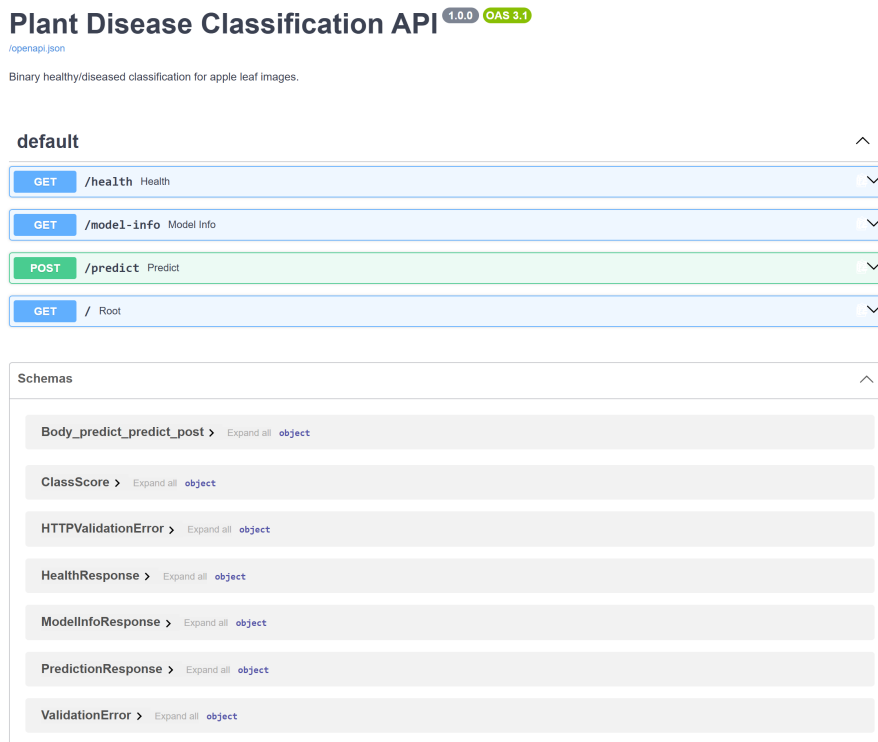


Figure 12. The generated OpenAPI documentation.

14. Challenges and how they were handled

Four problems were worth the time they took.

The first was recognising the leaf-grouping issue at all. The dataset's own card mentions that its split preserves leaf grouping, but the significance only becomes clear on counting: 3,171 images from 505 leaves is a mean of 6.28 photographs per leaf. The response was to build the split on leaf identity, audit the official split against the same rule, and assert the property in the test suite so it cannot regress silently.

The second was that the benchmark saturates. With validation macro F1 at 1.0000 from the third epoch onward, neither the architecture comparison nor the leakage demonstration could be settled on validation accuracy. The response was to measure leakage structurally instead, and to select the architecture on deployment cost, both of which are reported as such rather than dressed up as accuracy findings.

The third was training cost. Training runs on a four-core laptop CPU with no usable GPU. Benchmarking showed the model step, not the data loader, was the bottleneck - the loader needs about 13 seconds per epoch against roughly 127 for the model - so the fix was to reduce input resolution to 160 pixels and to use two loader workers rather than four, leaving the physical cores to the forward and backward pass instead of competing with it.

The fourth was the deployment target disappearing mid-project, described above.

15. Limitations

The following limitations are material to how these results should be read.

The benchmark is close to saturated. Validation macro F1 reaches 1.0000 by the third epoch, which says more about the dataset than about the model. Reported test accuracy of 99.51% should be read in that light.

PlantVillage is a laboratory dataset. Leaves are detached and photographed against uniform backgrounds under controlled lighting. Published work has repeatedly found that models trained on it degrade substantially on field photographs. Nothing in this project contradicts that finding, and nothing in this project measures it.

The model has only ever seen apple leaves. It will still return a confident healthy-or-diseased answer for a tomato leaf, a hand, or a photograph of a wall, because a two-class softmax has nowhere else to put its mass. There is no out-of-distribution detection.

The effective sample size is 505 leaves, not 3,171 images. The image count flatters the amount of independent evidence behind every figure in this report.

Confidence is uncalibrated, and the error analysis shows the model can be wrong at 0.99 confidence. The threshold flags uncertainty, not incorrectness.

Model selection used a saturated validation set, so the selected epoch is the first to reach the ceiling rather than a demonstrably better checkpoint.

The binary output hides which disease is present, which is what the brief asked for but is a real reduction: scab, black rot and cedar apple rust are different problems with different responses.

16. Future improvements

The most valuable next step would be evaluation on field imagery - PlantDoc or a similar in-situ dataset - to measure the laboratory-to-field gap rather than leaving it as a caveat. Everything else is secondary to knowing that number.

Beyond that: temperature scaling to calibrate confidence, together with a rejection option so that out-of-distribution uploads can be declined instead of confidently labelled; retaining the four-way disease head alongside the binary one, since the labels already carry that information; Grad-CAM overlays to check that the model attends to lesions rather than to background or leaf outline, which would also test the reading offered in the error analysis; and group-aware cross-validation across all 505 leaves in place of a single split, to report intervals rather than point estimates.

17. Conclusion

The system meets the brief: a transfer-learned classifier, a validated API, a light web interface, and a deployment path onto free infrastructure, with accuracy, precision, recall, F1 and a confusion matrix reported on a held-out test set.

The engineering judgement worth carrying forward is not the 99.51% accuracy figure. It is that this dataset contains 6.28 photographs of each physical leaf on average, that a conventional random split therefore leaks 100.0% of validation images, and that the resulting score would have looked slightly better while meaning considerably less. The reported number is lower than a leaky pipeline would have produced, and it is the one that can be defended.